

 Day 5 • 2 Hours

Advanced Python

Encapsulation (Protecting Data)

 **Goal:** Learn how to securely hide and protect object data to write robust applications.

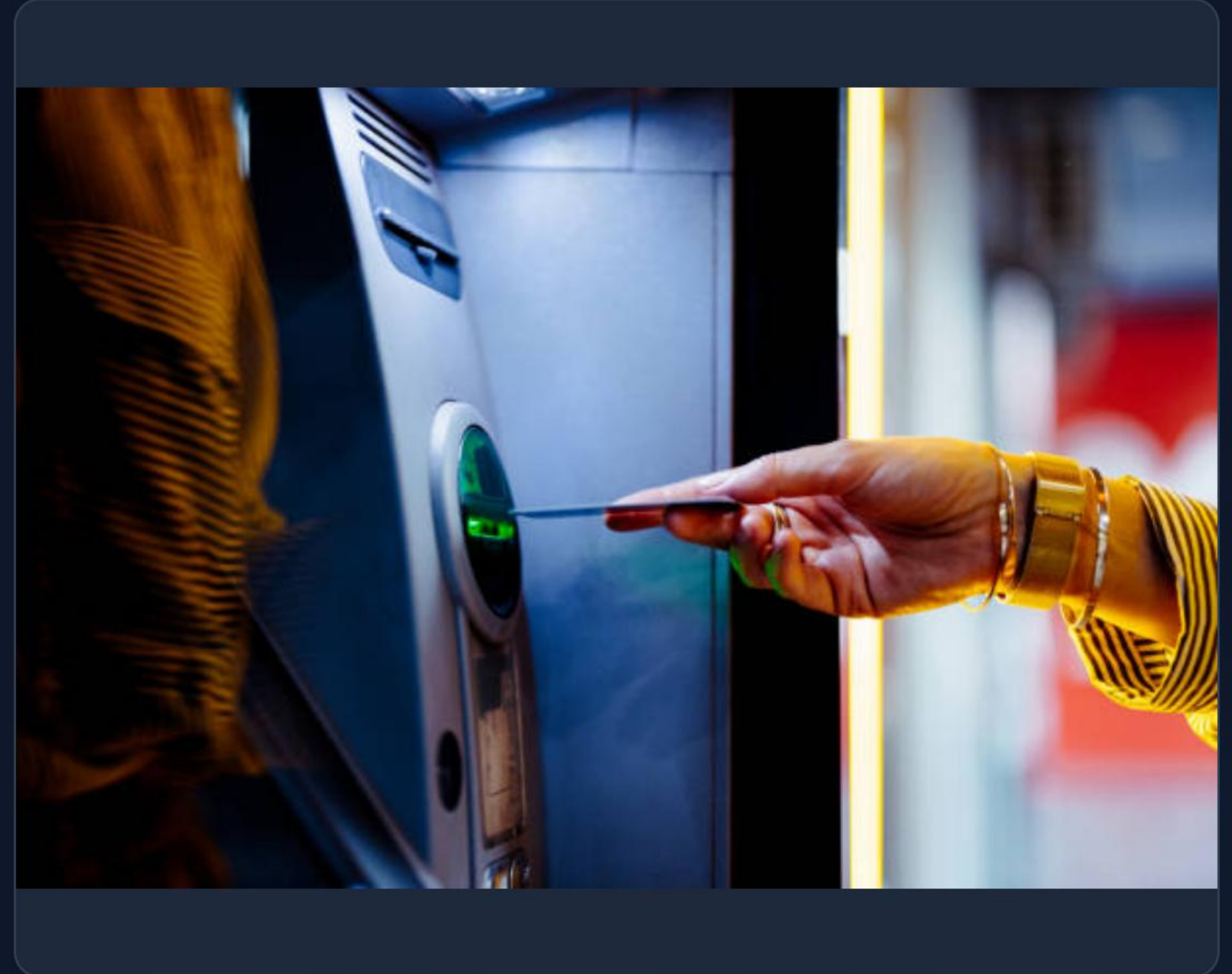
Real-Life Example — ATM Machine 🏦

When using an ATM:

- ✓ Check balance safely
- ✓ Withdraw money using set rules
- ✗ Cannot directly change the bank database
- ✗ You cannot set your balance manually

Why?

Because internal data is strongly protected. You can only interact through predefined physical and digital rules.



What is Encapsulation?



Hide Internal Data

Ensure that sensitive variables and the internal state of an object remain completely hidden from the outside world.



Controlled Access

Provide explicitly defined, safe methods (functions) that dictate exactly how outside code can interact with the data.



Protect Info

Prevent accidental or malicious changes that could corrupt the application's critical data and functionality.

Real Life → Python Mapping

Real World Concept	Python Implementation
Bank Vault	Private Variable
ATM Buttons	Methods (Functions)
Security Rules & Checks	Encapsulation Logic

Normal (Unsafe) Code ❌

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

acc = BankAccount(1000)
acc.balance = 1000000 # ⚠️ Unsafe
```

⚠️ The Problem

Anyone can change the balance directly without verification.

Without encapsulation, internal states are exposed to unexpected modifications, instantly breaking the security and logic of your application.

Protected Variable (_)

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

# _balance signals protected status
```

Developer Warning

A **single underscore** prefix _ means the variable is intended for internal use only.

Note: The data is technically still accessible, but directly modifying it is strongly discouraged by convention.

Private Variable (__) 🔥

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

acc = BankAccount(1000)
print(acc.__balance) # ❌ ERROR
```

🔒 Strict Protection

A **double underscore** prefix `__` makes the variable completely private to the class.

Trying to access it directly from the outside instantly throws an error. The data is securely locked down.

Access Using Methods (ATM Logic)

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def show_balance(self):
        print("Balance:", self.__balance)
```

Safe Interaction

Since the user cannot directly touch `__balance`, they must use the provided methods.

This allows the class to safely control how and when modifications happen, preventing corrupt states.

Real Meaning of Encapsulation 🔒

Controlled Access

User → presses ATM buttons

Program → controls rules internally

Encapsulation ensures that the internal state of an object is shielded from unauthorized access. The only way to interact with the data is through the explicitly provided, safe interfaces.



Instagram Example 📱

You cannot directly change:

- ✗ Followers count database
- ✗ Core password architecture

You must use app features.

This is the exact same idea as Python Encapsulation. The application provides verified methods (buttons, UI) to interact with hidden data securely.



Output Prediction

```
class Test:
    def __init__(self):
        self.__x = 10

t = Test()
print(t.__x)
```

Result:

 **ERROR: AttributeError**

Reason:

`__x` is explicitly declared as private and strictly cannot be accessed from outside the class.

Practice Task 1



Secure Wallet



Create class `Wallet`



Private money `__money`



Method `add_money()`



Method `check_money()`

Practice Task 2



Student Protection



Create class `Student`



Private `__marks`



Method `update_marks()`



Method `show_marks()`

Practice Task 3 & 4



Task 3: Mobile Lock System



Class `Phone`



Private `__password`



Method `unlock(password)`



Task 4: Game Health System



Class `Player`



Private `__health`



Method `take_damage()`



Method `heal()`

Why Companies Use Encapsulation



Without Encapsulation

- ✗ Bugs everywhere
- ✗ Severe security risks
- ✗ Unexpected data corruption



With Encapsulation

- ✓ Safe, isolated systems
- ✓ Highly controlled logic
- ✓ Clean, maintainable code

Industry Standard: Heavily used by Banks, WhatsApp, Amazon, and modern tech enterprises.

Day 5 Summary



- ✓ **Learned Encapsulation:** Core mechanism for securing object data.
- ✓ **Private Variables:** Understood how to completely hide attributes.
- ✓ **_ vs __:** Distinguished between protected (convention) and private (strict).
- ✓ **Access Methods:** Protected data by forcing interaction through safe rules.
- ✓ **Real-World Modeling:** Mapped physical security concepts to code structures.

You are now writing secure OOP code!

Image Sources



<https://media.istockphoto.com/id/1431478225/photo/close-up-photo-of-woman-hands-using-atm-machine.jpg?s=612x612&w=0&k=20&c=zh1hu87MB38qkrRbDYldVTkwC3lgm0sSMHyAhtUgKLg=>

Source: www.istockphoto.com



https://png.pngtree.com/thumb_back/fh260/background/20251124/pngtree-digital-security-concept-with-glowing-data-streams-connecting-padlocks-representing-secure-image_20576373.webp

Source: [pngtree.com](https://png.pngtree.com)



https://static.vecteezy.com/system/resources/thumbnails/070/528/667/small_2x/person-using-smartphone-interacting-with-social-media-icons-in-modern-office-setting-showing-digital-engagement-and-connection-photo.jpg

Source: www.vecteezy.com